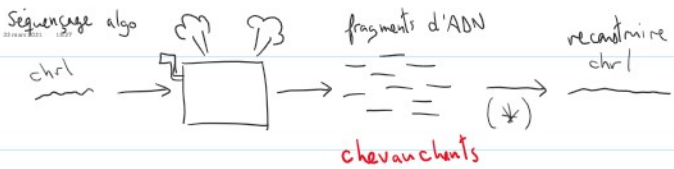
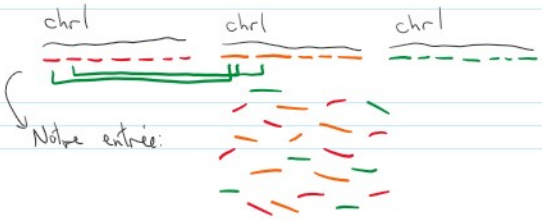




Problème:

<u>Entrée</u>: fragments $r_1, r_2, \dots, r_n$ proviennent de fragmentations différentes
chevauchants
appelés reads (lectures)

<u>Sortie</u>: séquence originale

- Problème de la + courte super-séquence

<u>Entrée</u>: $r_1, r_2, \dots, r_n$

<u>Sortie</u>: plus courte séquence S tq. chaque r_i est contenu dans S.

ex: $r_1 = ACCG$ $r_2 = CGCT$ $r_3 = GCTA$ $r_4 = GACC$

$S = GACCGCTA$

r_4 r_1 r_2 r_3

- Ce pb est NP-complet. (problème pas d'algo poly(n))
- Méthode de résolution classique
  - réduction au pb du commis voyageur (TSP)

- on définit $overlap(r_i, r_j)$ = taille du + long suffixe de r_i qui est un préfixe de r_j

ex: $r_i = ACGGT$ $r_j = GGTTTT$

$overlap(r_i, r_j) = 3$

ACGGT
GGTTTT → ACGGTTTT

- construire un graphe $G = (V, \mathcal{E})$ orienté

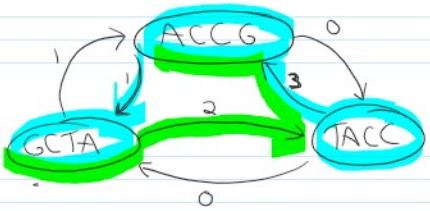
sommets → V ; $\mathcal{E}$ → arêtes (forme (u,v))

V: sommets: $\{r_1, r_2, \dots, r_n\}$

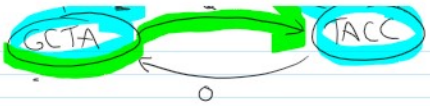
E: arête = $\{(r_i, r_j) : i, j \in \{1, \dots, n\}, i \neq j\}$

poids sur $(r_i, r_j) = overlap(r_i, r_j)$

$r_1 = ACCG$ $r_2 = GCTA$ $r_3 = TACC$



<u>But</u>: trouver un chemin qui passe exactement 1 fois par sommet et qui max. la somme des poids des arêtes.



poids des arêtes.

= TACC GCTA (algo glouton)
= GCTACCG (mieux)

└ Algo glouton :
- P = ∅
- commencer par l'arête (u,v) de poids max
- P.append(u)
- P.append(v)
- tant que |P| ≤ n
  - w = P.last()
  - trouvez z ∉ P qui max. le poids de (w, z)
  - P.append(z)
- return P

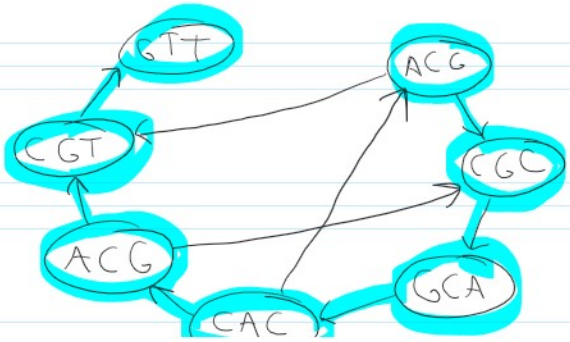
<u>Thm</u> : algo-glouton est une 4-approximation
sol glouton ≤ 4 · optimale

<u>Conjecture</u> : glouton ≤ 2 · optimale

Décomposition en k-mer
- k-mer : séq. de longueur k
- Lister tous les k-mer de chaque read (read long n, n-k+1 k-mers)

  r := ACCGT k = 3
  ↳ ACC CCG CGT
- Si chaque k-mer de la séq. originale S est présent, on devrait pouvoir reconstruire S avec seulement les k-mers.
- On va construire un graphe G = (V, E) où
  V = tous les k-mers (version avec ou sans répétition)
  E = { (X, Y) : X = aX' et Y = X'b , a, b ∈ Σ }
  chevauchement de taille k-1

  ex: k-mers ACG CGC GCA CAC ACG CGT GTT (avec répét.)



Graphe des k-mers
Idée : 2 k-mer consécut. fo = seq taille 4 (k+1)
ex: ACG → CGC = ACGC

<u>But</u> : trouver un chemin qui passe par chaque sommet 1 fois

[redacted] : ACGCACGTT

— chaque sommet 1 fois

- Quand même NP-complet
- Autre formalisation : #min de chemins disjoints pour couvrir chaque sommet

---

Graphes de Bruijn

- Autre modélisation
- Au lieu que sommet = k-mer
  → plutôt arête = k-mer
- Graphe $G = (V, E)$

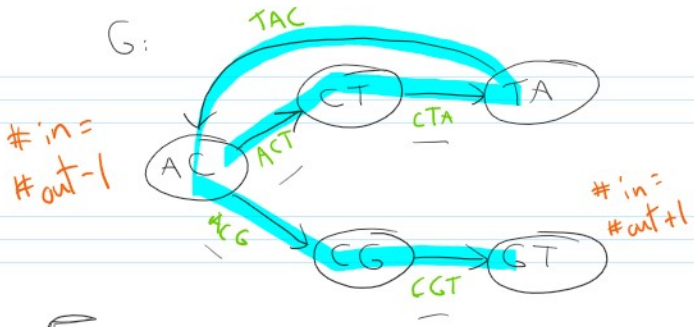
V = Tous les k-1 mers possibles qui apparaissent dans nos reads (sans répétition)

$E = \{ (X, Y) : \exists$ k-mer Z dans nos reads tel que $Z = aX$ et $Z = Yb$, $a, b \in \Sigma \}$

(avec répétition si multiples k-mers qui lient X et Y)

ex : Reads = CGT ACG CTA ACT TAC k=3

G :



arête = mes k-mers

<u>But</u> : trouver un chemin qui contient chaque arête 1 fois.

On appelle ceci un chemin Eulérien.

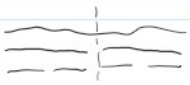
- On peut déterminer en <u>temps polynomial</u> s'il $\exists$ un chemin avec chaque arête 1 fois.

[redacted] : ACTACGT

#in = nb voisin entr.
#out = nb voisins sort.

- <u>Thm</u> : $\exists$ un chemin Eulérien si et seulement si
  - il y a ≤ 1 sommet tel que #in = #out - 1 ← départ
  - il y a ≤ 1 sommet tel que #in = #out + 1 ← arrivée
  - #in = #out pour tout autre sommet ← intermédiaire

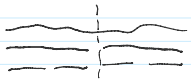
<u>Supposition</u> <u>Réalité</u>



Supposition

- chaque k-mer de la séq. originale est dans nos reads
- pas d'erreur dans nos reads
- solution unique

Réalité



- un k-mer peut être absent
- taux d'erreur variables sur nucl.
- plein de sds
  - └ segments fortement répétés

.... ACAAAACAAAA ...